

Universidad de Costa Rica
Escuela de Ingeniería Eléctrica

IE0435 — Inteligencia Artificial Aplicada a la Ingeniería Eléctrica
Proyecto 2 · I Ciclo 2026

Herramientas de predicción de demanda eléctrica

Integración de K-Nearest Neighbors y árboles de regresión al agente de series de tiempo

Gabriel Esquivel Nuñez— carné C22799
1 de julio de 2026

1. Contextualización del proyecto

El Centro Nacional de Control de Energía (CENCE) opera la curva de demanda eléctrica nacional en tiempo real y publica, junto con la demanda real, una demanda programada para las horas siguientes. La predicción precisa de esta demanda es esencial para la operación del Sistema Eléctrico Nacional: permite planificar el despacho de generación, reducir costos y mantener la estabilidad del sistema.

El presente proyecto parte de un agente de inteligencia artificial desarrollado previamente como proyecto eléctrico (repositorio de Erick Vargas Monge), construido sobre el enfoque de agentes con modelos de lenguaje local de Mauro Di Pietro. Dicho agente, ejecutado completamente en local mediante Ollama y sin claves de API, interpreta consultas en lenguaje natural y decide qué herramienta especializada ejecutar sobre un conjunto de datos de demanda cargado en memoria. Sus herramientas originales incluyen ejecución de código Python, graficación, y predicción mediante los modelos estadísticos Prophet y ARIMA/SARIMA.

El objetivo de este segundo proyecto fue ampliar las capacidades predictivas de ese agente incorporando dos herramientas adicionales basadas en aprendizaje automático clásico, según lo permitido en el enunciado del curso: predicción con K-Nearest Neighbors (regresión) y predicción con árboles de regresión.

2. Datos utilizados

Se trabajó con el conjunto de datos de demanda nacional descargado del portal del CENCE, ya depurado en el proyecto base (archivo datos_limpios.csv). Sus características son:

- 35 136 registros con frecuencia de muestreo de 15 minutos, cubriendo aproximadamente un año (de agosto de 2024 a agosto de 2025).
- Columna MW: potencia eléctrica real medida, en megavatios.
- Columna MW_P: potencia programada por el ICE para el mismo instante, que en este trabajo se usa como línea base (baseline) de referencia.
- Índice temporal fechaHora en formato datetime, con frecuencia fija de 15 minutos.

3. Herramientas desarrolladas

Ambas herramientas se integraron al flujo del agente respetando el mismo patrón de diseño del proyecto base: cada herramienta es una función de Python acompañada de un esquema de descripción en el formato de herramientas de Ollama, registrada en el diccionario central de herramientas, con normalización automática de los argumentos que produce el modelo de lenguaje, graficación del resultado y guardado automático en CSV.

3.1 Ingeniería de características

A diferencia de Prophet y ARIMA, que modelan internamente la tendencia y la estacionalidad de una serie temporal, KNN y los árboles de regresión son modelos en los que no se puede

agregar una noción de tiempo. Para habilitarlos como predictores de series temporales se construyó una matriz de características a partir de la propia serie:

- Rezagos (lags): los valores de la serie en los 4 intervalos previos (la última hora) y el valor del mismo instante 96 pasos atrás (un día completo). Aportan memoria de corto plazo y estacionalidad diaria.
- Variables de calendario: hora, minuto, día de la semana e indicador de fin de semana. Capturan el patrón de consumo que varía según la hora del día y el tipo de día.

La predicción a varios pasos se realiza de forma recursiva: cada valor predicho se reinyecta como rezago para predecir el siguiente intervalo, hasta cubrir el horizonte solicitado.

3.2 Herramienta predict_knn

Predice promediando los k casos históricos más parecidos en el espacio de características, ponderados por distancia. Dado que KNN se basa en distancias y las variables tienen escalas muy distintas (cientos de MW frente a horas de 0 a 23), se emplea un pipeline con estandarización previa (StandardScaler). Por defecto se usan 15 vecinos.

3.3 Herramienta predict_tree

Predice mediante un árbol de regresión que particiona recursivamente el espacio de características y asigna el promedio de cada hoja. No requiere escalado, pues es invariante a la escala de las variables. Para evitar el sobreajuste se limita la profundidad máxima del árbol y se exige un número mínimo de muestras por hoja. Su principal ventaja es la interpretabilidad: las particiones corresponden a reglas explícitas sobre la hora y el día.

4. Evaluación y resultados

Para validar las herramientas se implementó un procedimiento donde se entrena cada modelo con todo el histórico excepto el último día, se predice ese último día (96 intervalos de 15 minutos) de forma recursiva y se comparan las predicciones contra los valores reales mediante tres métricas: error absoluto medio (MAE), raíz del error cuadrático medio (RMSE) y error porcentual absoluto medio (MAPE). Como referencia se incluye la demanda programada del propio ICE (columna MW_P) sobre el mismo tramo.

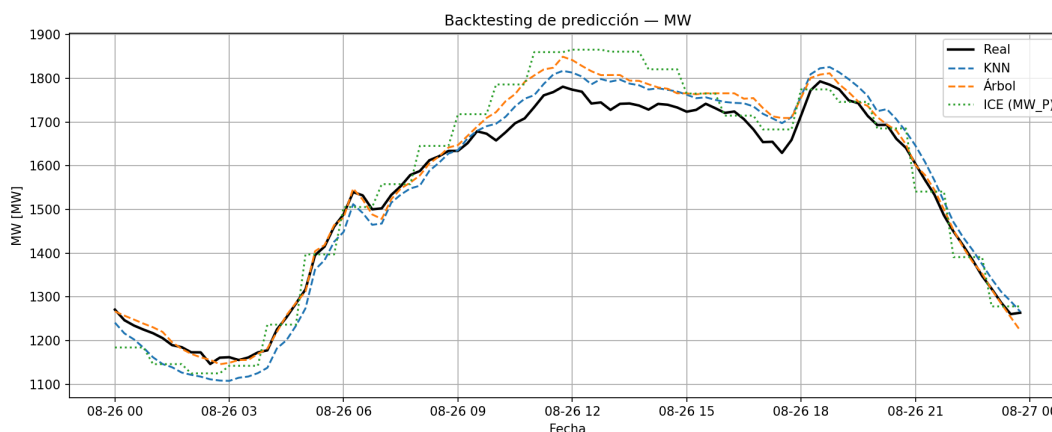


Figura 1. Demanda real frente a las predicciones de KNN, árbol de regresión y la programación del ICE en el día de prueba.

Los resultados obtenidos se resumen en la siguiente tabla:

Modelo	MAE	RMSE	MAPE (%)
Árbol de regresión	26.33	36.13	1.60
KNN (k = 15)	36.48	39.00	2.45
ICE (MW_P, baseline)	47.26	59.06	3.05

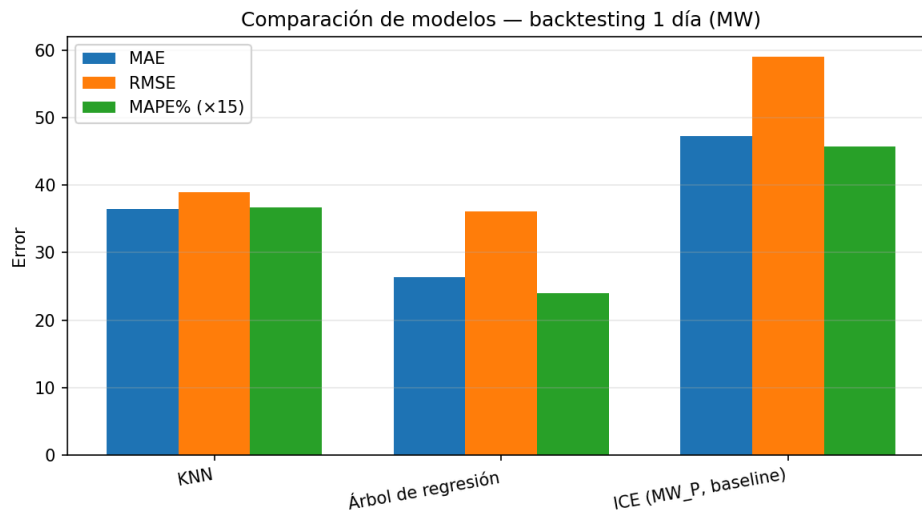


Figura 2. Comparación visual de las métricas de error por modelo (MAPE escalado $\times 15$ para visibilidad).

En la ventana evaluada, ambos modelos de aprendizaje automático superaron a la demanda programada del ICE: el árbol de regresión alcanzó el menor error en las tres métricas, con un MAPE de 1.60 % frente al 3.05 % del baseline. Conviene aclarar que estos valores corresponden a un único día de prueba y pueden variar según la fecha de corte; el procedimiento queda disponible en el módulo de evaluación para repetirse sobre otros tramos.

5. Integración con el agente

Las herramientas se conectaron al agente de modo que un usuario pueda invocarlas en lenguaje natural; por ejemplo, «predice MW para los próximos 2 días con KNN» o «predice la demanda de mañana usando un árbol de regresión». El prompt del sistema se amplió para describir las nuevas herramientas e incluir una guía breve sobre cuándo conviene cada modelo, y se agregó un normalizador de argumentos que traduce expresiones como «2 días» al número de pasos correspondiente y completa valores por defecto cuando el modelo de lenguaje los omite.

El conjunto resultante de herramientas de predicción del agente quedó conformado por cuatro técnicas complementarias: Prophet y ARIMA (estadísticas, del proyecto base) y KNN y árboles de regresión (aprendizaje automático, aporte de este proyecto).

6. Oportunidades de desarrollo futuro

- Incorporar variables mediante una API de acceso libre, en particular datos meteorológicos como lo es la temperatura, dado que la demanda eléctrica está fuertemente correlacionada con la temperatura por el uso de refrigeración.
- Sustituir el árbol único por modelos de conjunto (Random Forest o Gradient Boosting), que suelen reducir la varianza y mejorar la exactitud manteniendo la lógica de características aquí desarrollada.
- Añadir una herramienta de detección y corrección de anomalías que limpie automáticamente los datos antes de predecir, mejorando la robustez de todos los modelos.
- Ajustar automáticamente los hiperparámetros (número de vecinos, profundidad del árbol, conjunto de rezagos) mediante validación cruzada temporal, de modo que el agente seleccione la mejor configuración según el horizonte solicitado.
- Migrar el agente a un framework de código abierto (LangChain, LlamaIndex o CrewAI) para estandarizar la orquestación de herramientas y facilitar su mantenimiento.

7. Conclusiones

Se ampliaron exitosamente las capacidades del agente de predicción de demanda eléctrica con dos herramientas de aprendizaje automático integradas al mismo flujo conversacional del proyecto base. Los modelos que no son nativos de series temporales lograron predicciones competitivas, superando en la ventana evaluada a la demanda programada del operador del sistema. El procedimiento de backtesting incorporado permite cuantificar de forma objetiva el desempeño de cada técnica y sienta la base para las extensiones propuestas.

Repositorio y código

El código completo (herramientas, agente integrado y módulo de evaluación) acompaña este reporte. El proyecto base de referencia se encuentra en:

github.com/ErickVM7/Proyecto-Electrico